

Amlogic

Model Conversion User Guide

Revision: 1.5

Release Date: 2023-10-20

Copyright

© 2023 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 , and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time.

This product is not intended for use in medical, life saving, or life sustaining applications.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

变更记录

Issue1.5(2023-10-20)

第六版

1. 更新 adla_tool 支持的 Framework 版本信息
2. 新增 feature 量化算法以及对应参数说明：
 - (1) hist_percentile
 - (2) Hist_asymm_percentile
 - (3) history_moving-avg
3. 新增其他工具对应的介绍，包括：
 - (1) adla_info_tool
 - (2) quantize parameter search tool
 - (3) NNAPI_Optimize_tool

Issue1.4(2023-5-29)

第五版

1. 新增支持 uint8 量化转换
2. 新增支持导出 demo code 功能
3. 新增集成 adla_info 工具
4. 新增支持指定 ONNX、Tflite 模型的输出层

Issue 1.3 (2023-03-20)

第四版

1. 新增 feature 量化算法：
 - (1) kl_symm_divergence
 - (2) kl_asymm_divergence
 - (3) moving-avg、percentie
2. 新增 Feature 量化算法参数：
 - (1) --quantize-algo、--optimize-algo
 - (2) --kl-threshold-size、--divergence-nbins
 - (3) --percentile、--moving-alpha
 - (4) --neg-scale
3. 新增 Weights 量化算法：mle

4. 新增 Weights 量化算法参数:

- (1) --weights-quantize-algo、--weights-optimize-algo
- (2) --weights-threshold-size、--weights-neg-scale

Issue 1.2 (2022-11-21)

第三版

- 1. 更新工具版本对应的 Ubuntu 环境，添加工具包中文档说明
- 2. 新增参数--dtypes 使用说明
- 3. 新增 quantized tflite 转换指导
- 4. FAQ 章节新增模型转换错误和板测精度错误确认步骤

Issue 1.1 (2022-08-30)

第二版

- 1. 更新了各个 Framework 对应的版本
- 2. 新增模型转换流程图
- 3. 添加 int8 perlayer 的转换说明
- 4. 添加 Paddle 模型的转换命令

Issue 1.0 (2022-05-30)

初始版本.

目录

变更记录	ii
1. 简介	1
2. 工具介绍	2
2.1 简介	2
2.2 模型转换工具	2
2.2.1 工具结构	2
2.2.2 工具目录简介	2
2.2.3 环境依赖	3
3. 模型转换	5
3.1 模型转换简介	5
3.2 量化简介	5
3.2.1 量化详解	5
3.2.2 量化方法介绍	6
3.2.3 量化方式选取指导	6
3.3 模型转换流程图	7
3.4 模型转换	7
3.4.1 默认量化算法	7
3.4.2 其他量化算法	11
3.5 参数说明	17
3.5.1 简介	17
3.5.2 adla_convert 参数解析	17
4. 其他工具	21
4.1 Adla_info_tool 工具	21
4.2 Quantize parameter search tool	22
4.3 NNAPI_Optimize_tool	23
5. FAQ	24

1. 简介

本文将详细介绍使用 Aala_tool 工具进行模型转换的过程以及细节。指导公司内部或者外部相关开发人员进行模型转换。

当前我们的 Aala_tool 工具支持:caffe、pytorch、onnx、mxnet、darknet、tensorflow、keras、tflite、Paddle 等框架对应的模型。如果开发者使用的是其他框架模型，需先将模型转换成上述支持中的一种。

Confidential!
nick@khadas.com
2023-10-25 10:37:07

2. 工具介绍

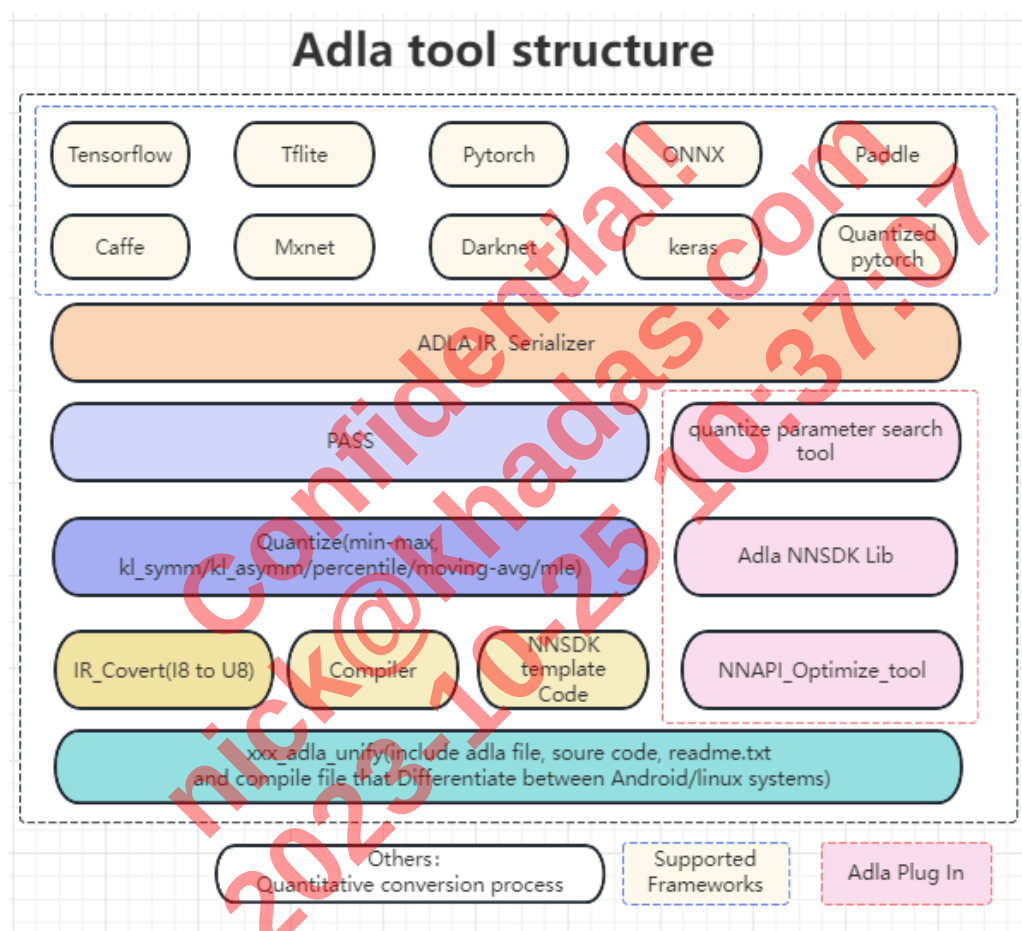
2.1 简介

当前将模型转换出 adla 文件需要的工具:

adla-toolkit 模型工具，用于将模型导入、量化、并转换出最终的 adla 文件。

2.2 模型转换工具

2.2.1 工具结构



2.2.2 工具目录简介

File In Adla_tool_binary-x.x.x	Description
bin/	Directory containing the executable file for adla tool
adla_convert	soft link to -> ./adlplib/adla_convert

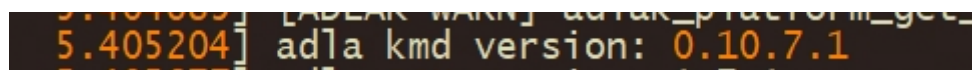
adlplib\	Directory containing the executable file for adla_tool
adla_convert	binary conversion command file,convert model to adla format
adla_plug_in\	Some plugins for adla tools
common\	Compiler and Configuration Files
template\	Template code and nnsdk package
android\	Template code for Android environment
linux\	Template code for Linux environment
example\	The demo code of nnsdk, will be readjusted in the future
nnsdk\	nnsdk development kit
tools\	some auxiliary tools
adla_info_tool\	adlainfo tool,parse adla file information
input_npy_txt_bin_generate.py	Generate random input npy/txt/bin file script
quantize_parameter_test\	Traversing quantization parameter scripts
demo/	Directory containing the samples for adla tool
convert_adla.sh	samples model conversion script
data/	Model quantization picture
dataset.txt	Model Quantization Dataset
model_source/	Models corresponding to each framework supported by adla
Readme.txt	Readme file
release_notes.txt	Release notes file

2.2.3 环境依赖

当前 adla_tool 是基于 Ubuntu20.04 python3.8.10 环境下编的 binary tool，客户只需保持基础环境是 Ubuntu20.04 python3.8.10 即可，不需要额外安装其他环境。

板侧 Driver 版本跟工具版本对应关系：

1. 重启板子获取串口内核日志，或者直接执行命令：`dmesg |grep version`



```
5.405204] adla kmd version: 0.10.7.1
```

- 2.adla 对应的 Driver version 为 0.10.7.1，当前只关注前三位即可。

- 3.Driver 版本号与 DDK 版本以及 adla_tool 版本的对应关系如下表格

DDK_Version	Driver Version	Adla_tool Version
DDK 1.2.2.4	0.10.2	adla-toolkit_0.9.3.6/1.0.0.6
DDK 1.4.3.5	0.10.4	adla-toolkit_1.2.0.7
DDK 1.7.5.5	0.10.7	adla-toolkit_1.2.0.9
DDK 2.1.1.6	1.2.0	adla-toolkit_1.5.10 adla-toolkit_1.6.10.3
DDK 2.3.6.7	1.3.0	adla-toolkit_2.0.11.0

Note:

- 1、Driver 版本号：第二步中获取到的版本号
- 2、Adla_tool version: 工具版本号，需与 Driver 版本匹配
- 3、DDK_Version: 只是一个对外 release 的 DDK 版本号

当前支持的 framework 版本以及模型加载方式为：

Framework	Version	Model load(Remark)
Tensorflow	2.11.1	Tf.compat.v1.lite.TFLiteConverter.from_frozen_graph()
Pytorch	1.12.0	torch.jit.load(model_path)
Onnx	1.12.0	onnx.load(model_path)
Mxnet	1.6.0	mx.model.load_checkpoint(prefix_path, prefix_epoch)
Darknet	https://github.com/siju-samuel/darknet	
Caffe	apt-get install caffe-cpu	caffe.Net(proto_file,blob_file,caffe.TEST)
tflite	2.10.0	tflite.Model.GetRootAsModel tflite.Model.Model.GetRootAsModel
Keras	2.11.0	tf.keras.models.load_model(model_path)
Paddle	2.4.2	Paddle.jit.load(model_path)

3. 模型转换

3.1 模型转换简介

模型转换过程是先将各个 framework 模型转换成统一格式的 ADLA IR，然后进行 int8、int16、uint8 量化，再转成基于我们平台能加载并运行的 adla 格式文件。

3.2 量化简介

量化：量化是将以往用 32bit 或者 64bit 表达的浮点数用 16bit、8bit 或者更低的 4bit 方式进行存储。

当前 ADLA 工具量化是复用 tensorflow 进行量化，支持 int8(Perlayer/Perchannel)、int16、uint8 三种量化方式。

3.2.1 量化详解

uint8：将 float32 数据量化成 unsigned int8 的数据。即：将最大/最小范围区间映射到 0~255 区间。

int8：将 float32 数据量化成 signed int8 的数据。即：将最大/最小范围区间映射到 -128~127 区间。

Int16：将 float 的数据量化成 signed int16 的数据。即：将最大/最小范围映射到 -32768 到 32767 区间。

int8/uint8 量化参数计算公式： (scale:映射比例, zero_point: 零点的位置)

$$\text{scale} = (\text{float_max_value} - \text{float_min_value}) / (\text{quantized_max} - \text{quantized_min})$$
$$\text{zero_point} = \text{quantized_max} - \text{float_max_value} / \text{scale}$$

Int16 量化参数计算公式： (scale:映射比例)

$$\text{float_max_temp} = \max(|\text{float_max_value}|, |\text{float_min_value}|)$$
$$\text{scale} = \text{float_max_temp} / \text{quantized_max}(32767)$$
$$\text{zero_point} = 0$$

例如：某个节点输出的最小/最大值范围为 (-4~1.076)

uint8 量化参数计算：

$$\text{scale} = (1.076 - (-4)) / (255 - 0) = 0.0199$$
$$\text{zero_point} = 255 - 1.08 / \text{scale} = 201$$

int8 量化参数计算：

$$\text{scale} = (1.076 - (-4)) / (127 - (-128)) = 0.0199215$$
$$\text{zero_point} = 127 - 1.076 / \text{scale} = 73$$

Int16 量化参数计算：

$$\text{float_max_temp} = \max(|-4|, |1.08|)$$
$$\text{scale} = 4 / 32767 = 0.000122074$$

3.2.2 量化方法介绍

模型量化包括 Feature 量化和 Weights 量化。

Feature 量化方法：

1. MinMax 量化 (Default)：直接从 fp32 张量中选取最大最小值来确认动态范围
2. kl_symm_divergence: KL 对称量化算法，将 float32 数值分布和 int8 数值分布抽象成两个分布，用阈值 |T| 来更新这两个数值分布，并用 KL 散度来衡量这两个分布的相似性。
3. Kl_asymm_divergence: KL 非对称量化，将 float32 数值分布和 int8 数值分布抽象成两个分布，用[Min,Max]来更新这两个数值分布，同时使用一个缩放因子 scale 来缩小 [Min,Max]范围，并用 KL 散度来衡量这两个分布的相似性。
4. moving-avg: 滑动平均最大最小值。
5. Percentile: 基于百分比系数来调整 fp32 张量中最大最小值的动态范围。该算法对 Feature 数据分布存在个别极大的离群点数值的模型更为有效。
6. hist_percentile: 直方图百分比系数量化算法
7. hist_asymm_percentile: 直方图非对称百分比系数量化算法
8. history_moving-avg: 历史系数滑动平均量化算法

Weights 量化类型：per-channel 和 per-layer。

Weights Per-layer 量化：conv2d/Dwconv2d 等算子的 filter tensor 量化时统计一个 min/max 并进行量化，per-layer 量化能减小在单板上运行时占用的 DDR 内存和带宽。

Weights Per-channel 量化：conv2d/Dwconv2d 等算子的 filter tensor 量化时，根据 filter 个数分别统计对应的 min/max 值并进行量化，每一个 filter 对应一组量化参数，per-channel 量化能提升量化后的精度。特别对一些 conv 算子的 filter 范围分布差异较大的模型，使用 per-channel 量化，精度提升明显。

Weights 量化方法：当前有 MinMax 和 MLE 量化方法。

1. MinMax 量化 (Default)：直接从 fp32 张量中选取最大最小值来确认动态范围
2. MLE:最大似然估计量化方法，对 perlayer、perchannel 量化均有效。

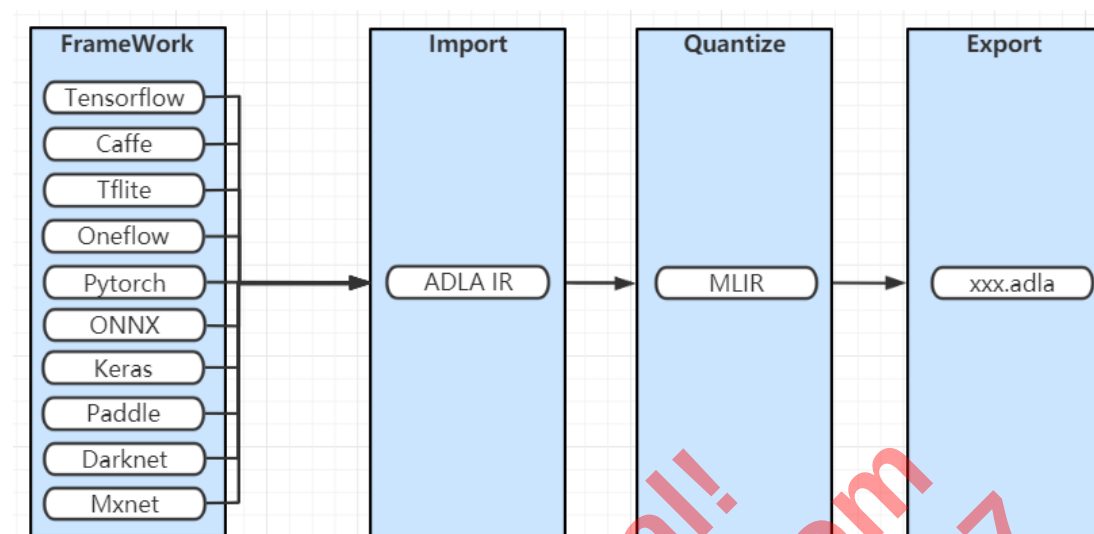
3.2.3 量化方式选取指导

量化方式选取规则：

1. 一般量化类型建议使用当前默认值，即：int8 perlayer 量化
2. 如果 int8 perlayer 量化的精度不够，可以设置成 perchannel 量化。或者使用 int16 量化。
3. 当前 feature 和 weights 都默认使用的是 MinMax 量化算法。Feature 算法和 weights 算法在某些参数条件下能提升模型精度，weights 和 feature 量化算法分别平均能提升 0.5%左右的精度。每个模型的最优量化精度对应的参数均不一致，需要客户遍历量化参数，并进行量化后的精度验证来找出最优结果。Feature+weights 算法结合起来对精度的提升效果也需遍历更多参数才能获得。

4. Feature 量化算法选择跟 weights 的 per-layer/perchannel 量化无关

3.3 模型转换流程图



3.4 模型转换

当前模型转换过程都是在 adla-toolkit 目录下进行的。可执行文件均在 bin 目录下。

3.4.1 默认量化算法

1、Feature/weights Min-Max 量化:

Caffe/Darknet:

```

./bin/adla_convert --model-type caffe(darknet) \
--model xx.prototxt(xxx.cfg) \
--weights xx.caffemodel(xxx.weights) \
--quantize-dtype int8(int16/uint8) \
--source-file dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--disable-per-channel False(default True) \
--export-template-code True(default True) \
--system-env 'android' \
#--batch-size 4(default 1)
  
```

Onnx/Pytorch:

```

./bin/adla_convert --model-type onnx(tflite) \
--model ./xxx.onnx(xxx.tflite) \
--quantize-dtype int8(int16/uint8) \
--inputs "0 1 2" \
--input-shapes "3,288,288#3,288,288#3,288,288" \
--source-file dataset.txt \
--channel-mean-value "128,128,128,128#0,0,0,255#0,0,0,255" \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--batch-size 1 \
--disable-per-channel False(default True) \
--export-template-code True(default True) \
--system-env 'android' \
# --outputs "output0 output1" #onnx/tflite supports specifying output nodes
#--shape-with-batch False#False#False \
  
```

```
--dtypes float32#float32#float32"
```

Mxnet:

```
./bin/adla_convert --model-type mxnet \
--model ./mobilenet0.25-symbol.json \
--weights ./mobilenet0.25-0000.params \
--quantize-dtype int8(int16/uint8) \
--inputs "data" \
--input-shape "3,224,224" \
--channel-mean-value "0,0,0,255" \
--source-file dataset.txt \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--export-template-code True(default True) \
--system-env 'android' \
--batch-size 4(default 1)
```

Tensorflow:

```
./bin/adla_convert --model-type tensorflow \
--model xxxx.pb \
--quantize-dtype int8(int16/uint8) \
--inputs data \
--input-shape 112,112,3 \
--outputs fc1/mul \
--channel-mean-value "0,0,0,255" \
--source-file dataset.txt \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--export-template-code True(default True) \
--system-env 'android'
```

Keras:

```
./bin/adla_convert --model-type keras \
--model xxxx.h5 \
--inputs input1 \
--quantize-dtype int8(int16/uint8) \
--input-shape "3,224,224" \
--channel-mean-value "0,0,0,255" \
--source-file dataset.txt \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--export-template-code True(default True) \
--system-env 'android'
```

Quantized Tflite:

```
./bin/adla_convert --model-type quantized_tflite \
--model resnet18.tflite \
--target-platform PRODUCT_PID0XA001 \
--outdir test
```

Pytorch:

```
./bin/adla_convert --model-type pytorch \
--model xxxx.pt \
--quantize-dtype int8(int16/uint8) \
--inputs data \
--input-shape 224,224,3 \
--channel-mean-value "0,0,0,255" \
--source-file dataset.txt \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
```

```
--export-template-code True(default True) \
--system-env 'android' \
```

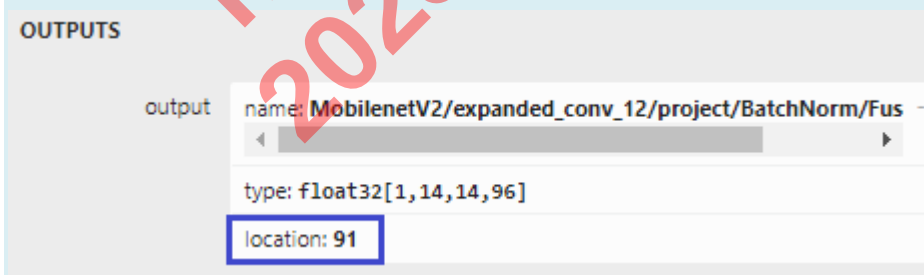
Paddle

```
./bin/adla_convert --model-type paddle \
--model xxxxx \
--quantize-dtype int8(int16/uint8) \
--inputs data \
--input-shape 224,224,3 \
--channel-mean-value "0,0,0,255" \
--source-file dataset.txt \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--export-template-code True(default True) \
--system-env 'android' \
```

注:

1. `--model-type`: 转换的模型类型, 当前支持: `onnx`, `pytorch`, `caffe`, `darknet`, `mxnet`, `keras`, `quantized_tflite`, `tflite`, `tensorflow`, `paddle`, `oneflow` 等
2. `--model` (模型文件)/`--weights` (模型权值文件): `--model` 必须设置参数。 `model-type` 为 `caffe/darknet/mxnet` 时需要设置 `--weights` 参数
3. `--inputs/--input-shapes`: 在 `model-dtype` 不为 `quantized_tflite/caffe/darknet` 时需要设置
4. `--inputs`: 输入节点名称。如果有多个输入, 请使用空格分隔每个输入, 例如 “`input1 input2`”
5. `--outputs`: 输出节点名称。如果有多个输出, 请使用空格分隔每个输出, 例如 “`output1 output2`” 当前 `outputs` 在 `model-dtype` 为 `tensorflow` 时必须设置。在 `model-type` 为 `onnx/tflite` 时, 默认不需要设置, 但支持设置中间层为输出节点。

`Onnx` 模型直接设置中间层的 `OUTPUTS` 对应的 `name` 即可。`Tflite` 模型设置的是中间层 `OUTPUTS` 对应的 `location number`。使用 `Netron` 工具可以查看 `onnx` 模型的 `OUTPUTS name` 和 `tflite` 模型的 `OUTPUTS` 对应的 `location number`, `tflite locat number` 的如下图:



6. `--input-shapes`: 输入节点的尺寸。如果有多个输入, 请使用#分隔每个输入的尺寸, 例如: “`3, 224, 224#3, 299, 299#3, 12, 12`”
7. `--shape-with-batch`: 设置的输入 `shape` 是否带 `batch` 信息(可选项)。默认值: `False`。如果设置为 `True`, 则模型实际输入就是 `input-shapes` 设置的尺寸。如果有多个输入, 请用#分隔开。例如, 模型输入为: “`1x224x224x3`”, “`11x88`” 时, 参数设置为: `--input-shapes “224, 224, 3#11, 88” --shape-with-batch “False#True”`

8. `--dtypes`: 输入类型, 设置输入对应的 type 信息(可选项)。默认值: float32。如果有多个输入, 请用#分隔, 例如: `--dtypes : "float32#float32#int8"`

9. `--quantize-dtype`: 量化类型, 当前支持 int8、int16、uint8 三种量化类型

当量化类型为 int8/int16 时, 需要设置: `--source-file/--channel-mean-value` 这组参数。但如果 `source-file` 是 npy 文件时, 则不需要设置 `channel-mean-value`。

10. `--source-file dataset.txt`. txt 文件中提供量化图片的路径。当前支持图片和 npy 两种格式。如: `./data/cat.jpg` 或者 `./data/input_3_224_224.npy`

一般建议提供 500~1000 张左右的图片来量化。如果为多输入模型时, `dataset.txt` 中将一组输入对应的图片放一行并以空格间隔开, 如一组输入: `./1.jpg ./2.jpg ./3.jpg`。如果量化数据有多组时, txt 文件中一行设置一组输入, 设置多行即可。npy 文件做量化数据时, shape 需要跟 `input-shapes` 参数设置的保持一致。

11. `--channel-mean-value`: 根据训练模型时的预处理方式来设置该预处理参数。参数包括四个值 (`m1, m2, m3, scale`)。前三个值为均值参数, 最后一个值为 `scale` 参数。对于输入为三通道数据 (`data1, data2, data3`), 预处理过程为:

$$Out1 = (data1 - m1) / scale$$

$$Out2 = (data2 - m2) / scale$$

$$Out3 = (data3 - m3) / scale$$

例如: 如果前处理需将数据归一化到 $[-1, 1]$ 之间, 参数设置为: `128, 128, 128, 128`

如果前处理需将参数归一化到 $[0, 1]$ 之间, 参数设置为: `0, 0, 0, 255`

如果是单通道模型, 参数设置方式为: `m1, 0, 0, scale`

如果是多输入模型, 可以针对多个输入设置不同的 `channel-mean-value`, 以#号隔开, 例如: `--channel-mean-value "0, 0, 0, 1#127.5, 127.5, 127.5, 127.5#0, 0, 0, 255"`

如果图片三通道对应的 `scale` 均不相同或者模型是非图片输入时, 建议先使用 Python 读取模型的输入数据并做完减 mean 除以 `scale` 的操作后保存成 npy 文件, 在模型转换时, 量化的 `source-file` 中提供 npy 文件路径即可

12. `--batch-size`: 设置转换出 adla 文件的 `batch-size`, 当前默认值为 1

13. `--iterations`: 可选参数, 默认值 1000, 如果 `dataset.txt` 中的提供的是多组数据, 且需要使用所有提供的数据进行量化时, 请设置 `--iterations` 参数, 确保 `iterations*batch-size=多组数据个数`

14. `--disable-per-channel`: 是否禁用逐通道量化方式, 默认为 True。设置 True 时, 为 perlayer 量化, 否则为 perchannel 量化。

15. `--outdir`: 转换出的 adla 文件输出路径, 默认值为当前路径下的 "xxx_adla_unify" 目录

16. `--target-platform`: 指定转换目标平台的 adla 文件。当前默认值: `PRODUCT_PID0XA001`

当前有如下有效值:

`PRODUCT_PID0XA001`、`PRODUCT_PID0XA002`

`PRODUCT_PID0XA003`、`PRODUCT_PID0XA004`

可以使用如下方式来确认应该设置的值, 执行: `cat /proc/cpuinfo`

查看倒第二行数 `Serial` 对应的数列码前四个字符 (`Serial` 后面粗体字部分)

序列号和参数 `PRODUCT_PID0X?` 的对应关系如下图:

PID	Serial
0XA001	Serial: 3d0a010100000000c613492931563343
0XA002	Serial: 3e0a0202000000007c404b6c31563553
0XA003	Serial: 360c01030000000025f904ab31563541
0XA004	Serial: 420a010100000000daff6e4931583354

例如:

在串口或者 cmd 窗口执行命令: `cat /proc/cpuinfo` 后, Serial 如下图

```
Serial       : 360c010300000000946bd13930563754
Hardware     : Amlogic
```

此时, 参数应该设置为: `--target-platform PRODUCT_PID0XA003`

Note:

1. 如果没有获取到对应上表格中的序列号, 可以执行: `cat /proc/cpu_chipid`
2. 如果序列号的前四位没有匹配上, 可以只匹配前三位

17. `--export-template-code`: 是否导出 demo code, 默认为 True。设置为 False 时, 只转换出 adla 文件

18. `--system-env`: 指定单板系统环境, 默认为 linux。跟 `--export-template-code` 配套使用。使用默认参数时, 导出的 demo code 中带有 linux 对应的编译脚本。设置为 "android" 时, 导出的 demo code 中带有 Android 对应的编译脚本。当前支持 [linux, android] 两个有效值, 用户根据当前单板环境来设置。然后参考生成的 demo code 中的 Readme.txt 的指导进行编译即可。

结果: 最终生成带有 adla 文件的 demo code, 如下图:

```
resnet34-v2-7_adla_unify/
├── CMakeLists.txt
├── config.h
├── main.c
├── Readme.txt
└── resnet34-v2-7_int8.adla
```

3.4.2 其他量化算法

一般使用上一章节中的默认量化算法即可。如果默认量化算法的精度不满足需要, 可以尝试使用进阶量化算法。

当前适配了多种量化算法, 不同量化算法对应的参数各不相同, 下面会列出不同量化算法的使用方法。客户如果需要使用进阶量化算法, 建议二客户参照 4.2 章节中介绍的 Quantize parameter search tool 工具来使用进阶量化算法。

2、Feature kl_symm_divergence+weights Min-max 量化

以 caffe model 为例:

```
./bin/adla_convert --model-type caffe(darknet) \
--model xx.prototxt(xxx.cfg) \
--weights xx.caffemodel(xxx.weights) \
--quantize-dtype int8(int16/uint8) \
--source-file dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir convert_output \
```



```
--target-platform      PRODUCT_PID0XA001 \
--disable-per-channel  False \
--quantize-algo         "kl_symm_divergence" \
--optimize-algo         "kl-dis" \
--kl-threshold-size     8192 \
--divergence-nbins      16384 \
--export-template-code  True(default True) \
--system-env 'android'
```

注:

- 跟默认的 MinMax 量化相比, KL 对称量化新增了四个参数, 其他参数的设置跟 MinMax 量化时一致
- quantize-algo: 默认值为:"min-max", 当前设置量化算法类型为 "kl_symm_divergence"。
- optimize-algo: 度量 float32 数值分布和 int8 数值分布的相似性的方法。默认值: "kl-dis", 设置 kl_symm_divergence 时, 对应的有效值为:"kl-dis"/"mse-dis"
- kl-threshold-size: KL 量化时 Feature 是否使用 kl 量化的阈值。如果量化的 Feature size 过小, 不宜使用 KL 量化。默认值: 2048, 一般建议直接使用默认值。
- divergence-nbins: KL 量化时, 直方图中 bin 的个数。默认值: 2048, 建议取值: 2048/4096/8192/12288/16384

部分模型使用 kl 对称量化算法, 使用不同参数进行量化后的精度如下图: (第一行结果为 min-max 量化对应精度)

quantize_algo	optimize_algo	num_histogram_bins	kl_thread_size	resnet_18		resnet_34		googlenet-7		shufflenet-v2-10	
				TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
kl_symm_divergence	NA	NA	NA	62.72%	84.64%	67.24%	88.22%	67.50%	88.20%	49.98%	72.98%
	kl-dis	2048	2048	63.22%	84.66%	67.46%	88.24%	67.36%	88.44%	46.06%	70.60%
			4096	63.26%	84.54%	67.82%	88.28%	67.64%	88.20%	46.14%	70.72%
			8192	62.60%	84.72%	67.12%	88.00%	67.56%	88.40%	46.36%	70.82%
			12288	62.60%	84.72%	67.12%	88.00%	67.28%	88.36%	48.62%	72.08%
			16384	62.60%	84.72%	67.12%	88.00%	67.34%	88.36%	48.62%	72.08%
	mse-dis	2048	2048	62.92%	84.74%	67.18%	87.96%	67.22%	88.38%	45.38%	69.50%
			4096	62.38%	84.70%	66.78%	87.88%	67.20%	88.36%	50.30%	72.86%
			8192	62.38%	84.70%	66.78%	87.88%	67.12%	88.34%	50.30%	72.86%
			12288	62.38%	84.70%	66.78%	87.88%	66.96%	88.42%	49.84%	72.88%
			16384	62.38%	84.70%	66.78%	87.88%	67.20%	88.32%	49.84%	72.88%
		4096	2048	62.52%	84.72%	67.14%	88.00%	67.16%	88.28%	50.30%	72.86%
			4096	62.52%	84.72%	67.14%	88.00%	67.40%	88.32%	50.30%	72.86%
			8192	62.52%	84.72%	67.14%	88.00%	67.22%	88.24%	50.30%	72.86%
			12288	62.52%	84.72%	67.14%	88.00%	67.20%	88.16%	49.84%	72.88%
			16384	62.52%	84.72%	67.14%	88.00%	67.26%	88.28%	49.84%	72.88%
		8192	2048	62.60%	84.76%	67.10%	88.12%	67.34%	88.32%	49.66%	72.64%
			4096	62.60%	84.76%	67.10%	88.12%	67.16%	88.12%	49.66%	72.64%
			8192	62.60%	84.76%	67.10%	88.12%	67.42%	88.20%	49.66%	72.64%
			12288	62.60%	84.76%	67.10%	88.12%	67.28%	88.06%	49.86%	73.04%
			16384	62.60%	84.76%	67.10%	88.12%	67.34%	88.18%	49.86%	73.04%
		12288	2048	62.86%	84.74%	67.12%	87.92%	67.40%	88.20%	50.16%	72.66%
			4096	62.86%	84.74%	67.12%	87.92%	67.48%	88.24%	50.16%	72.66%
			8192	62.86%	84.74%	67.12%	87.92%	67.26%	88.30%	50.16%	72.66%
			12288	62.86%	84.74%	67.12%	87.92%	67.38%	88.22%	49.94%	73.00%
			16384	62.86%	84.74%	67.12%	87.92%	67.40%	88.34%	49.94%	73.00%
		16384	2048	62.68%	84.56%	67.00%	88.08%	67.06%	88.16%	49.94%	72.86%
			4096	62.68%	84.56%	67.00%	88.08%	67.42%	88.36%	49.94%	72.86%
			8192	62.68%	84.56%	67.00%	88.08%	67.14%	88.06%	49.94%	72.86%
			12288	62.68%	84.56%	67.00%	88.08%	67.10%	88.22%	50.14%	72.96%
			16384	62.68%	84.56%	67.00%	88.08%	67.46%	88.30%	50.14%	72.96%

3、Feature Kl_asymm_divergence+weights Min-max 量化

以 caffe model 为例:

```
./bin/adla_convert --model-type      caffe(darknet) \
--model                          xx.prototxt(xxx.cfg) \
--weights                        xx.caffemodel(xxx.weights) \
--quantize-dtype                  int8(int16/uint8) \
--source-file                     dataset.txt \
--channel-mean-value              "128,128,128,128" \
--outdir                          convert_output \
--target-platform                  PRODUCT_PID0XA001 \
--disable-per-channel              False \
--quantize-algo                    "kl_asymm_divergence" \
--optimize-algo                    "kl-dis" \
--kl-threshold-size                2048 \
--divergence-nbins                 2048 \
--neg-scale                        0.9 \
--export-template-code             True(default True) \
--system-env                       'android'
```

注:

- 跟默认的 MinMax 量化相比, KL 对称量化新增了四个参数, 其他参数的设置跟 MinMax 量化时一致
- quantize-algo: 默认值为"min-max", 当前设置量化算法类型为 "kl_symm_divergence"。
- optimize-algo: 度量 float32 数值分布和 int8 数值分布的相似性的方法。默认值: "kl-dis", 设置 kl_symm_divergence 时, 对应的有效值为: "kl-dis"/"js-dis"/"cos-dis"
- kl-threshold-size: KL 量化时 Feature 是否使用 kl 量化的阈值。如果量化的 Feature size 过小, 不宜使用 KL 量化。默认值: 2048, 一般建议直接使用默认值。
- divergence-nbins: KL 量化时, 直方图中 bin 的个数。默认值: 2048, 建议取值: 2048/4096/8192/12288/16384
- neg-scale: 默认值: 1.0

部分模型使用 Feature 的 percentile 算法量化后精度结果如下图: (首行数据为 feature min-max 精度)

quantize_algo	optimize_algo	histogram_bin	kl_threshold_size	neg_scale	Number of pictures for	resnet_18		shufflenet-v2-10		squeezenet1.1-7		mnasnet1.0		Darknet19	
NA	NA	NA	NA	NA	200	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
kl_asymm_divergence	cos-dis	2048	4096	0	200	62.72%	84.64%	49.98%	72.98%	50.14%	74.92%	48.66%	73.16%	72.92%	90.92%
				0.001	200	62.98%	84.74%	49.56%	73.02%	50.40%	74.98%	48.58%	73.20%	73.24%	91.38%
				0.01	200	63.08%	84.96%	49.68%	73.04%	50.06%	74.50%	48.74%	73.32%	73.28%	91.20%
				0.1	200	62.92%	84.66%	49.44%	72.96%	50.06%	74.50%	48.66%	73.20%	73.26%	91.30%
				1	200	62.96%	84.70%	49.92%	73.20%	50.06%	74.50%	48.58%	73.24%	73.26%	91.06%
				1.2	200	63.08%	84.96%	50.00%	72.82%	50.06%	74.50%	48.94%	73.12%	73.66%	91.40%
				1.5	200	62.96%	84.58%	49.82%	73.10%	50.06%	74.50%	48.66%	73.12%	73.42%	91.48%
				0	200	63.08%	84.96%	49.58%	72.86%	50.06%	74.50%	48.88%	73.30%	73.50%	91.46%
				0.001	200	62.84%	84.74%	49.78%	73.14%	50.54%	74.84%	48.84%	73.16%	73.30%	91.30%
				0.01	200	62.84%	84.76%	49.84%	72.98%	50.06%	74.50%	48.66%	73.04%	73.50%	91.24%
				0.1	200	63.08%	84.96%	49.92%	72.94%	50.06%	74.50%	48.78%	73.34%	73.22%	91.36%
				1	200	63.10%	84.66%	49.62%	73.20%	50.06%	74.50%	49.14%	73.16%	73.40%	91.14%
				1.2	200	63.08%	84.96%	49.82%	72.94%	50.06%	74.50%	48.88%	73.16%	73.34%	91.24%
				1.5	200	63.08%	84.96%	50.00%	73.14%	50.06%	74.50%	48.88%	73.34%	73.40%	91.34%
				0	200	63.08%	84.96%	49.58%	72.86%	50.06%	74.50%	48.88%	73.30%	73.40%	91.36%
				0.001	200	62.96%	84.64%	49.14%	72.96%	50.72%	75.08%	48.64%	73.32%	73.36%	91.20%
				0.01	200	63.02%	84.78%	50.18%	72.88%	50.06%	74.50%	48.80%	73.32%	73.28%	91.36%
				0.1	200	63.08%	84.96%	49.90%	73.12%	50.06%	74.50%	48.76%	73.24%	73.28%	91.30%
				1	200	63.08%	84.96%	49.88%	72.88%	50.06%	74.50%	48.52%	73.22%	73.42%	91.06%
				1.2	200	63.08%	84.96%	49.90%	72.92%	50.06%	74.50%	48.94%	73.38%	73.16%	91.30%
				1.5	200	63.34%	84.80%	50.12%	72.98%	50.06%	74.50%	48.90%	73.28%	73.40%	91.26%
				0	200	63.08%	84.96%	49.66%	73.02%	50.06%	74.50%	48.68%	73.38%	73.36%	91.28%

4、Feature Percentile+weights Min-max 量化

以 caffe model 为例:

```
./bin/adla_convert --model-type      caffe(darknet) \
--model                          xx.prototxt(xxx.cfg) \
--weights                        xx.caffemodel(xxx.weights) \
--quantize-dtype                  int8(int16/uint8) \
--source-file                     dataset.txt \
--channel-mean-value              "128,128,128,128" \
```

```

--outdir          convert_output \
--target-platform PRODUCT_PID0XA001 \
--disable-per-channel False \
--quantize-algo    "percentile" \
--percentile       0.9999 \
--export-template-code True(default True) \
--system-env 'android'
# --asymm-percentile 0.999

```

注:

- 跟默认的 MinMax 量化相比, Percentile 量化新增了两个参数, 其他参数的设置跟 MinMax 量化时一致
- quantize-algo: 默认值为:"min-max", 当前设置量化算法类型为 "percentile"。
- percentile: 百分比系数, 默认值: 0.9999, 建议取值范围: **0.99 ~ 1.0**
- asymm-percentile: 非对称百分比系数, 默认值: -1.0, 建议取值范围: 0.99 ~ 1.0. 当为默认值时, 为对称百分比量化算法。当设置为其他值时, 为非对称百分比量化算法。

部分模型使用 Feature 的 percentile 算法量化后精度结果如下图: (首行数据为 feature min-max 精度)

quantize_algo	optimize_algo	num_histogram_bins	kl_thread_size	number of pictures for Quant	resnet_18		resnet_34		googlenet-7		shufflenet-v2-10		squeezenet1.1-7	
					TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
percentile	NA	NA	NA	100	62.72%	84.64%	67.24%	88.22%	67.50%	88.20%	49.98%	72.98%	50.14%	74.92%
	NA	NA	0.9991	100	62.84%	84.63%	66.92%	87.88%	67.02%	88.33%	48.86%	72.62%	50.40%	74.80%
	NA	NA	0.9992	100	62.98%	84.69%	66.76%	87.90%	67.08%	88.33%	48.72%	72.60%	50.22%	74.64%
	NA	NA	0.9993	100	62.90%	84.74%	66.88%	88.12%	67.18%	88.33%	48.74%	72.64%	50.42%	74.70%
	NA	NA	0.9994	100	62.92%	84.70%	66.98%	87.86%	67.16%	88.30%	49.44%	72.66%	50.60%	74.66%
	NA	NA	0.9995	100	63.16%	84.72%	67.14%	87.94%	67.26%	88.28%	49.52%	73.06%	50.18%	75.06%
	NA	NA	0.9996	100	63.04%	84.62%	67.10%	88.10%	67.22%	88.50%	49.40%	72.60%	50.50%	74.78%
	NA	NA	0.9997	100	63.20%	84.64%	67.18%	88.06%	67.24%	88.40%	49.86%	73.14%	50.38%	74.32%
	NA	NA	0.9998	100	63.06%	84.66%	66.98%	88.06%	67.62%	88.24%	49.58%	72.84%	50.12%	74.50%
	NA	NA	0.9999	100	62.88%	84.72%	66.96%	88.36%	67.48%	88.32%	49.82%	72.98%	50.04%	74.68%

5、Feature Moving-avg+weights Min-max 量化

以 caffe model 为例:

```

./bin/adla_convert --model-type    caffe(darknet) \
--model              xx.prototxt(yyy.cfg) \
--weights            xx.caffemodel(yyy.weights) \
--quantize-dtype      int8(int16/uint8) \
--source-file         dataset.txt \
--channel-mean-value  "128,128,128,128" \
--outdir              convert_output \
--target-platform     PRODUCT_PID0XA001 \
--disable-per-channel False \
--quantize-algo        "moving-avg" \
--moving-alpha        0.2 \
--export-template-code True(default True) \
--system-env 'android'

```

注:

- 跟默认的 MinMax 量化相比, moving-avg 量化新增了两个参数, 其他参数的设置跟 MinMax 量化时一致
- quantize-algo: 默认值为:"min-max", 当前设置量化算法类型为 "moving-avg"。
- moving-alpha: moving-alpha 系数, 默认值: 0.2, 建议取值范围: **0.2~0.9**

部分模型使用 Feature 的 moving-avg 算法量化后精度结果如下图: (首行数据为 feature min-max 精度)

quantize_algo	optimize_algo	num_histogram_bins	kl_thread_size	number of pictures for Quant	resnet_18		resnet_34		googlenet-7		shufflenet-v2-10		squeezenet1.1-7	
					TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
NA	NA	NA	NA	100	62.72%	84.64%	67.24%	88.22%	67.50%	88.20%	49.98%	72.98%	50.14%	74.92%
moving-avg	NA	0.2	NA	100	60.46%	84.56%	64.80%	88.18%	67.08%	88.30%	49.08%	73.18%	50.34%	74.34%
	NA	0.3	NA	100	61.30%	84.70%	65.54%	88.28%	67.10%	88.42%	49.60%	72.64%	50.22%	74.44%
	NA	0.4	NA	100	61.42%	84.50%	66.00%	88.12%	67.06%	88.26%	49.66%	73.00%	50.78%	74.28%
	NA	0.5	NA	100	61.78%	84.78%	66.08%	87.98%	67.28%	88.44%	49.78%	72.82%	49.94%	73.82%
	NA	0.6	NA	100	62.16%	84.54%	66.46%	87.96%	67.46%	88.32%	49.76%	73.12%	50.06%	74.76%
	NA	0.7	NA	100	62.42%	84.70%	66.82%	87.98%	67.40%	88.42%	49.98%	72.72%	50.42%	74.44%
	NA	0.8	NA	100	62.80%	84.62%	66.56%	87.98%	67.32%	88.52%	50.28%	72.98%	50.00%	74.26%
	NA	0.9	NA	100	62.86%	84.54%	67.00%	88.00%	67.32%	88.30%	49.72%	73.36%	49.98%	74.82%

6、Feature history_moving-avg + weights Min-max 量化

以 caffe model 为例:

```
.bin/adla_convert --model-type caffe(darknet) \
--model xx.prototxt(xxx.cfg) \
--weights xx.caffemodel(xxx.weights) \
--quantize-dtype int8(int16/uint8) \
--source-file dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--disable-per-channel False \
--quantize-algo "history_moving-avg" \
--moving-alpha 0.2 \
--export-template-code True(default True) \
--system-env 'android'
```

注:

1. history_moving-avg 的参数跟 moving-avg 算法的参数一致
2. --quantize-algo: 默认值为"min-max",当前设置量化算法类型为 "history_moving-avg"。

7、Feature hist_percentile/hist_asymm_percentile + weights Min-max 量化

以 caffe model 为例:

```
.bin/adla_convert --model-type caffe(darknet) \
--model xx.prototxt(xxx.cfg) \
--weights xx.caffemodel(xxx.weights) \
--quantize-dtype int8(int16/uint8) \
--source-file dataset.txt \
--channel-mean-value "128,128,128,128" \
--outdir convert_output \
--target-platform PRODUCT_PID0XA001 \
--disable-per-channel False \
--quantize-algo "hist_percentile" \ # "hist_asymm_percentile"
--percentile 0.9999 \
--export-template-code True(default True) \
--system-env 'android'
```

注:

1. hist_percentile/hist_asymm_percentile 的参数一致, 只需要设置百分比系数--percentile 即可
2. --quantize-algo 当前设置量化算法类型为"hist_percentile" 或者 "hist_asymm_percentile"

8、Feature Min-Max+Weights MLE 量化

以 caffe model 为例:

```
./bin/adla_convert --model-type      caffe(darknet) \
--model                          xx.prototxt(xxx.cfg) \
--weights                        xx.caffemodel(xxx.weights) \
--quantize-dtype                  int8(int16/uint8) \
--source-file                     dataset.txt \
--channel-mean-value              "128,128,128,128" \
--outdir                          convert_output \
--target-platform                  PRODUCT_PID0XA001 \
--disable-per-channel              True \
--weights-quantize-algo            "mle" \
--weights-optimize-algo            "cos-dis" \
--weights-threshold-size           2048 \
--weights-neg-scale                0.999 \
--export-template-code             True(default True) \
--system-env                       'android'
```

注:

- 跟 weights 默认的 MinMax 量化相比, mle 量化新增了四个参数, 其他参数的设置跟 MinMax 量化时一致
- weights-quantize-algo: 默认值为:"min-max",当前设置量化算法类型为"mle".
- weights-optimize-algo: 默认值(cos-dis),当前有效值为: "cos-dis"/"kl-dis"/"js-dis"
- weights-neg-scale: weights 量化时的缩放系数, 默认值(0.999),当前可设置范围:0.99~1.

部分模型使用 Weights 的 MLE 算法量化后精度结果如下图: (首行数据为 min-max 精度)

Weights quantize algo	optimize algo	weights thread_size	Number of pictures for Quantize	weights neg_scale	resnet18-v1-7		resnet34-v2-7		resnet50-v2-7		shufflenet-v2-10		squeezenet1.1-7	
Min-Max	NA	NA	200	NA	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5	TOP1	TOP5
MLE	cos-dis	2048	200	0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	49.94%	72.50%	49.98%	74.10%
				0.991	64.98%	87.26%	71.28%	89.90%	71.60%	90.30%	51.36%	74.14%	49.44%	74.58%
		16384	200	0.999	65.22%	87.44%	70.84%	89.94%	72.12%	90.12%	49.92%	72.98%	49.32%	74.18%
				0.99	64.88%	87.48%	71.12%	90.12%	71.54%	89.90%	51.60%	73.98%	49.98%	74.10%
	js-dis	2048	200	0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	49.94%	72.50%	49.98%	74.10%
				0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	51.96%	74.08%	49.98%	74.10%
		16384	200	0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	49.94%	72.50%	49.98%	74.10%
				0.99	64.88%	87.48%	70.94%	89.94%	71.54%	89.90%	49.94%	72.50%	49.98%	74.10%

9、Feature kl_asymm_divergence + Weights MLE 量化

以 caffe/darknet model 为例:

```
./bin/adla_convert --model-type      caffe(darknet) \
--model                          xx.prototxt(xxx.cfg) \
--weights                        xx.caffemodel(xxx.weights) \
--quantize-dtype                  int8 \
--source-file                     dataset.txt \
--channel-mean-value              "128,128,128,128" \
--outdir                          convert_output \
--target-platform                  PRODUCT_PID0XA001 \
--quantize-algo                    kl_asymm_divergence \
--optimize-algo                    cos-dis \
--kl-threshold-size                4096 \
--divergence-nbins                 4096 \
--neg-scale                        0.9 \
--disable-per-channel              True \
--weights-quantize-algo            "mle" \
--weights-optimize-algo            "cos-dis" \
--weights-threshold-size           2048 \
--weights-neg-scale                0.999 \
--export-template-code             True(default True) \
--system-env                       'android'
```


3.5 参数说明

3.5.1 简介

3.4.1 章节中介绍的模型转换过程中的参数是最简洁的使用参数，一般情况下只使用上述参数即可正常转换出 adla 文件。本章节 adla_convert 转换命令的所有参数进行扩展说明，以供使用。

3.5.2 adla_convert 参数解析

```
usage: adla_convert.py [-h] --model-type MODEL_TYPE --model MODEL
                        [--weights WEIGHTS] [--inputs INPUTS]
                        [--outputs OUTPUTS] [--input-shapes INPUT_SHAPES]
                        [--shape-with-batch SHAPE_WITH_BATCH]
                        [--dtypes DTYPES]
                        [--batch-size BATCH_SIZE] [--iterations ITERATIONS]
                        [--outdir OUTDIR] [--quantize-dtype QUANTIZE_DTYPE]
                        [--source-file SOURCE_FILE]
                        [--channel-mean-value CHANNEL_MEAN_VALUE]
                        [--disable-per-channel DISABLE_PER_CHANNEL]
                        [--inference-input-type INFERENCE_INPUT_TYPE]
                        [--inference-output-type INFERENCE_OUTPUT_TYPE]
                        [--target-platform TARGET_PLATFORM]
                        [--quantize-algo QUANTIZE_ALGO]
                        [--optimize-algo OPTIMIZE_ALGO]
                        [--kl-threshold-size KL_THRESHOLD_SIZE]
                        [--divergence-nbins DIVERGENCE_NBINS]
                        [--percentile PERCENTILE]
                        [--asymm-percentile ASYMM_PERCENTILE]
                        [--moving-alpha MOVING_ALPHA]
                        [--neg-scale NEG_SCALE]
                        [--weights-quantize-algo WEIGHTS_QUANTIZE_ALGO]
                        [--weights-optimize-algo WEIGHTS_OPTIMIZE_ALGO]
                        [--weights-threshold-size WEIGHTS_THRESHOLD_SIZE]
                        [--weights-neg-scale WEIGHTS_NEG_SCALE]
                        [--export-template-code EXPORT_TEMPLATE_CODE]
                        [--system-env SYSTEM_ENV]
```

Argument	Description
--model-dtype(required)	Requires a string value to denote the type of the convert model. Ranges: caffe, onnx, keras, darknet, tensorflow, quantized_tflite, tflite, pytorch, onnx.
--model(required)	Requires a string value to denote the file path of the imported model file, such as xxx.prototxt, xxx.cfg, xxx.onnx
--weights	Requires a string value to denote the file path of the imported weights file, such as xxx.caffemodel, xxx.weights, xxx.params.
--inputs	Multiple points are separated with spaces. For example, "input1 input2"
--input-shapes	Tensor shape sizes of the same input points are separated with commas. For example, '3,224,224', which represents the tensor shape sizes of one input point.

	Sizes between different input points are separated with hashtags. For example, '3,224,224#3,299,299#12,1', tensor shape sizes of three input points.
--shape-with-batch	Describes whether the --input-shapes contains the highest batch dimension or not. (Optional) None (Default) means --input-size-list should be given without batch. Enclose the bool values in quotation marks, and use # to separate the bool values. Take a graph with three input layer as an example, the three input layers with shape "?x224x224x3", "11x88", "10", input-shapes and shape with batch could be given as: --input-shapes "224,224,3#11,88#10" --shape-with-batch "False#True#True"
--dtypes	Describes the input types (Optional) float32 (Default) Enclose the string values in quotation marks, and use # to separate the bool values. Effective value["float32",'int64','int32','bool','uint8'] Take a graph with three input layer as an example, the three input layers with shape "?x224x224x3", "11x88", "10", input-shapes and shape with batch could be given as: --input-shapes "224,224,3#11,88#10" --shape-with-batch "False#True#True" --dtypes "float32#int32#int64"
--outputs	Multiple points are separated with spaces. For example, 'output1 output2'. Note: The current outputs must be set when the model-dtype is tensorflow. When the model-type is onnx/tflite, it does not need to be set by default, but it supports setting the middle layer as an output node.. The Onnx model can directly set the name corresponding to the OUTPUTS of the middle layer. The Tflite model sets the location number corresponding to the middle layer OUTPUTS.
--batch-size	Batch size value. Default value:1
--iterations	Number of iterations to run, integer value. (Optional) 1000 (Default)
--outdir	Requires a string value to denote the output path of the generated adla file, default: "xxx_adla_unify" (current folder)
--quantize-dtype	Data type used for quantized data. (Optional) Valid dtype values are: int8 : signed 8bit, default per-channel, if per-layer quantization is required, use it with disable-per-channel parameter int16 : signed 16bit, default per-channel, if pe-rlayer quantization is required, use it with disable-per-channel parameter uint8 : unsigned 8bit
--source-file	Dataset path and filename. The file extension indicates the dataset: xxx.txt, set the image path or npz file path in txt.
--channel-mean-value	Input channels mean value. This parameter should always be given according to running models. Note: This parameter is only valid for int8/int16 quantization and the source-file is image path .

--disable-per-channel	default True
--inference-input-type	After converting to adla, the input data type of the model. Default: same as quantize-dtype For int8 quantize, inference input type only supports float32, int8, uint8 . For int16 quantize, inference input type only supports float32, int16 .
--inference-output-type	After converting to adla, the output data type of the model. Default: same as quantize-dtype For int8 quantize, inference input type only supports float32, int8, uint8 . For int16 quantize, inference input type only supports float32, int16 .
--target-platform	Specify the config file to export adla file on the target platform. valid values as below: PRODUCT_PID0XA001(default) PRODUCT_PID0XA002 PRODUCT_PID0XA003 PRODUCT_PID0XA004
--quantize-algo	1. " kl-asymm_divergence ": The kl asymmetric quantization algorithm, If this algorithm is selected, you can set the parameters related to the algorithm: --optimize-algo, --kl-threshold-size, --divergence-nbins, --neg-scale 2. " kl-symm_divergence ": The kl symmetric quantization algorithm, If this algorithm is selected, you can set the parameters related to the algorithm: --optimize-algo, --kl-threshold-size, --divergence-nbins 3. " percentile ": The percentile algorithm, a percentile-based activation clamping quantification method. If this algorithm is selected, you can set the parameters related to the algorithm: --percentile 4. " moving-avg ": The moving-avg algorithm. If this algorithm is selected. You can set the parameters related to the algorithm --moving-alpha
--optimize-algo	Quantitative methods to measure the similarity between real and quantified data distributions, only valid for " kl_symm_divergence "/" kl_asymm_divergence " Valid optimize-algo values are: 1. " kl-dis ": Default optimize algorithm, kl divergence 2. " js-dis ": js divergence, only valid for " kl_asymm_divergence " 3. " cos-dis ": cosine similarity, only valid for " kl_asymm_divergence " 4. " mse-dis ": mean square error loss, only valid for " kl_symm_divergence "
--kl-threshold-size	Whether the feature uses the kl quantization threshold during kl quantization. If the feature size of the corresponding layer > threshold_size, use it. only valid for " kl_symm_divergence "/" kl-asymm_divergence ". 2048(default) The integer must equal 2N where N is a positive integer, recommended values: 2048/4096/8192/12288/16384
--divergence-nbins	only valid for " kl_symm_divergence "/" kl-asymm_divergence "

	Requires an integer to specify the number of bins in the Kullback-Laiber divergence (KL divergence) histogram. 2048(default) The integer must equal 2N where N is a positive integer, Recommended value: 2048/4096/8192/12288/16384
--percentile	Only valid for " percentile ", 0.9999(default) Suggested value range: 0.999 ~ 0.9999 , step_size 0.0001
--asymm-percentile	Only valid for "percentile", -1.0(default) Suggested value range: 0.999 ~ 0.9999 , step_size 0.0001 When it is the default value, it is a symmetric percentage quantization algorithm. When set to other values, it is an asymmetric percentage quantization algorithm.
--moving-alpha	only valid for " moving_alpha ", 0.2(default) Suggested value range: 0.2 ~ 0.9, step_size 0.1
--neg-scale	Scale factor for asymmetric quantization. only valid for " kl-asymm_divergence ", 1.0(default) Suggested value range: 0.001/0.01/0.1/1/1.2/1.5
--weights-quantize-algo	The weights quantization algorithm. Valid values are: 1. " min-max ": Default algorithm 2. " mle ": The mle quantization algorithm,
--weights-optimize-algo	Quantitative methods to measure the similarity between real and quantified data distributions, only valid for " mle " Valid optimize-algo values are: 1. " kl-dis ": Default optimize algorithm, kl divergence 2. " js-dis ": js divergence, 3. " cos-dis ": cosine similarity
--weights-threshold-size	The weights shape size threshold for mle quantization of weights. 2048(default) recommended values: 2048/4096/8192/12288/16384
--weights-neg-scale	Scale factor for weights mle quantization. only valid for " mle ", 0.999(default) Suggested value range: 0.99~1.0
--export-template-code	Whether to export demo code default True
--system-env	Specify the single board system environment. Used with --export-template-code, just to export compiled scripts in different environments. valid values as below: " linux ": default " android "

4. 其他工具

4.1 Adla_info_tool 工具

Adla_info 是一个能解析生成的 adla 文件的工具。通过 adla_tool 工具，可以获取到 adla 文件对应的平台信息、compiler 版本信息以及输入输出等信息。

Adla_info tool 集成在工具中的路径：

adla-toolkit-binary-x.x.x.x/bin/adla_plug_in/tools/adla_info_tool/ 下，包含下面几个文件：

```
adla-toolkit-binary-1.6.10.3/bin/adla_plug_in/tools/adla_info_tool/  
├── adlainfo_release  
├── mobilenet_v1_1.0_224_quant_u8.adla  
├── README_release_EN.txt  
└── README_release.txt
```

客户可以根据 README_release.txt 中的介绍来使用 adla_tool 工具。一般常用命令：

1、./adlainfo_release -v mobilenet_v1_1.0_224_quant_u8.adla 可获取 adla 文件对应的版本信息和平台信息

```
adla file name : mobilenet_v1_1.0_224_quant_u8.adla  
*****  
Compile_Version : 0.5.3  
Aml Hw Version  : 1.0  
*****
```

2、./adlainfo_release -io mobilenet_v1_1.0_224_quant_u8.adla，获取 adla 文件对应的模型输入输出信息以及 config 中配置的 AXI_SRAM 大小

```

adla file name : mobilenet_v1_1.0_224_quant_u8.adla
*****
Inputs                                     : [88 ]
  Input[0]                                : 88
  Dim Count                               : 4
  Size of dim[0]                           : 1
  Size of dim[1]                           : 224
  Size of dim[2]                           : 224
  Size of dim[3]                           : 3
  type                                     : Uint8
  scale                                    : [0.007812 ]
  zero_point                               : [128 ]
  inputs_memory_size                       : 150528
*****
Outputs                                    : [87 ]
  Output[0]                               : 87
  Dim Count                               : 2
  Size of dim[0]                           : 1
  Size of dim[1]                           : 1001
  type                                     : Uint8
  scale                                    : [0.003906 ]
  zero_point                               : [0 ]
  outputs_memory_size                     : 1001
*****
config:
  max_outstanding_outputs                 : 4
  axi_sram_size                           : 262144 bytes
*****

```

4.2 Quantize parameter search tool

Quantize parameter search tool 工具是基于 Adla_tool 工具添加了一些列 feature/weights 量化算法的基础上开发的一个最优量化参数搜索工具。

Quantize parameter search tool 集成在工具中的路径：

adla-toolkit-binary-x.x.x.x/bin/adla_plug_in/tools/quantize_parameter_test/ 下，包含下面几个文件：

```

quantize_parameter_test/
├── Model_quantize_guide.txt
└── Quantize_parameter_test.py

```

客户可根据该目录下的 Model_quantize_guide 的介绍来使用该工具。

当前工具是在模型转换过程中，将量化后的 IR 导出成 quant tflite 模型，再通过运行 quant tflite 来确认量化效果的。当前工具中的判断标准是通过统计跟原始模型的单帧输出结果的余弦相似度和欧氏距离来判断选取最优值的。

目前已使用该工具提升了多个客户的模型的量化精度效果。

工具使用建议：

- 1、如果客户的模型有其他的校验集，建议在 run_tflite 接口里面，将检验代码集成进去，根据客户自己的校验结果搜索最优的量化参数。

2、搜索建议：1.修改脚本中 `self.quantize_algos=['min-max']`,此时会遍历不同数据集的量化效果。优先选取一个在 min-max 量化下的最优量化参数 iterations。2.修改 `self.iteration=200` 的值为第一步中搜索到的最优值，再设置 `self.quantize_algos=['kl_asymm_divergence','kl_symm_divergence','percentile','moving-avg','mle']`,进行第二轮搜索。

4.3 NNAPI_Optimize_tool

NNAPI_Optimize_tool 是针对 NNAPI runtime 对于部分算子不支持导致模型拆分从而影响运行效率的问题，开发了一套 对于引入拆分的算子进行重构并在 nnapi 处理流程进行 pass 优化以提升模型运行效率的算子重构工具。

NNAPI_Optimize_tool 集成在工具中的路径：

`adla-toolkit-binary-x.x.x.x/bin/adla_plug_in/tools/tflite_refactorer_tool/` 下，包含下面几个文件：

```
tflite_refactorer_tool/  
├── dped_instance_quant.tflite  
├── Readme_CN.txt  
├── Readme_EN.txt  
└── tflite_refactorer
```

客户可根据该目录下的 Readme 的介绍来使用该工具。只有在客户需要通过 NNAPI 流程加载运行 tflite 模型时才需要使用该工具。

5. FAQ

1、模型转换出错问题如何定位

(1) 通过当前错误 log 进行初步确认

模型转换失败，一般会有对应的错误 log。先 check 下错误 log 信息，看是否能看出问题点。如果不能进行下一步。

(2) 确认错误出现在哪一步

在转换 log 中搜索关键字：Adla_tool，当前转换过程中，搜索关键字出现的结果有下面几种：

I Adla_tool: step_0 enter, begin to load original model.

I Adla_tool: step_0 out, load original model successfully.

I Adla_tool: step_1 enter, frontend begin to parse model.

I Adla_tool: step_1 out, frontend parse model successfully.

I Adla_tool: step_2 enter, begin to convert model to ir.

I Adla_tool: step_2 out, convert model to ir successfully.

I Adla_tool: step_3 enter, begin to quantize model.

I Adla_tool: step_3 out, quantize model successfully.

I Adla_tool: step_4 enter, begin to convert adla_file.

I Adla_tool: step_4 out, convert adla_file successfully, out_result is:
paddle_output/mobilenet_v1_int8.adla !

通过确认该流程 log，能初步确认当前是在哪一步处理中出现异常

(3) 不同位置出错的处理方法

Step_0: load original model，如果模型搜索关键字之后，只出现了 step_0 enter 的 log，那表明原始模型加载失败，此时需要客户根据 2.2 章节表格中各个 framework 对应的版本以及模型加载方式确认原始模型是否存在问题。

例如：下图中 step_0 enter, begin to load original model 之后没有加载成功的 log 提示

```

I Namespace(batch_size=1, channel_mean_value=None, default_ranges_max=None, default_ranges_min=None,
input_shapes='3,640,640', inputs='input', iterations=1, mean=0, model='yolox.pt', model_type='pytorch',
e_source_file=None, std_dev=1, target_platform='PROPUCT_PID0XA001', weights=None)
Adla_tool: step_0 enter, begin to load original model.
E
Unknown type name 'NoneType':
Serialized File "code/_torch_/torchvision/extension.py", line 1
def _assert_has_ops() -> NoneType:
~~~~~ <--- HERE
    _0 = _torch_.torchvision.extension._has_ops
    _1 = "Couldn't load custom C++ ops. This can happen if your PyTorch and torchvision versions are i
on on the compatible versions, check https://github.com/pytorch/vision#installation for the compatibi
on with torchvision._version_ and verify if they are compatible, and if not please reinstall torchv
'_assert_has_ops' is being compiled since it was called from 'nms'
Serialized File "code/_torch_/torchvision/ops/boxes.py", line 22
    scores: Tensor,
    iou_threshold: float) -> Tensor:
~~~~~ <--- HERE
    _6 = _torch_.torchvision.extension._assert_has_ops
    _7 = _6()
    _8 = ops.torchvision.nms(boxes, scores, iou_threshold)
'nms' is being compiled since it was called from '_batched_nms_coordinate_trick'
Serialized File "/home/dengliu/anaconda3/envs/python3.8/lib/python3.8/site-packages/torchvision/ops/boxes.py",
offsets = idxs.to(boxes) * (max_coordinate + torch.tensor(1).to(boxes))
boxes_for_nms = boxes + offsets[:, None]
keep = nms(boxes_for_nms, scores, iou_threshold)
~~~~ <--- HERE
    return keep
Serialized File "code/_torch_/torchvision/ops/boxes.py", line 16
    _5 = torch.unsqueeze(torch.slice(offsets), 1)
    boxes_for_nms = torch.add(boxes, _5)
    keep = _torch_.torchvision.ops.boxes.nms(boxes_for_nms, scores, iou_threshold, )
~~~~~ <--- HERE
    _0 = keep
    return _0
'_batched_nms_coordinate_trick' is being compiled since it was called from 'YOLOX.forward'
Serialized File "code/_torch_/mmdet/models/detectors/yolox.py", line 11
def forward(self: _torch_.mmdet.models.detectors.yolox.YOLOX,
img: Tensor) -> Tuple[Tensor, Tensor]:
    _0 = _torch_.torchvision.ops.boxes._batched_nms_coordinate_trick
~~~~~ <--- HERE
    _1 = _torch_.torchvision.ops.boxes._batched_nms_coordinate_trick
    _2 = _torch_.torchvision.ops.boxes._batched_nms_coordinate_trick
I -----warning(0)-----

```

Step_1:前端解析模型出错，可将错误 log 信息以及模型提供给 Amlogic 进行 debug

Step_2:转换 IR 时出错，可以设置 export ADLA_LOG_LEVEL=DEBUG 看是否有更多的错误信息，可先和 Amlogic 确认下 log 提示信息是否明确，如果 log 信息不足以支持 debug，需考虑将模型提供给 Amlogic 进行 debug。

Note: ADLA_LOG_LEVEL 有 DEBUG、INFO、WARNING、ERROR 四个级别，默认 INFO 级别

Step_4: 最后一步编译器转 adla_file 出错，可先和 Amlogic 确认下 log 提示信息是否明确。如果 log 信息不足以支持 debug，需要考虑将模型提供给 Amlogic 进行 debug。

2、板上运行结果精度差，如何定位？

(1) 确认 float 结果与原始平台精度是否一致

当前工具支持将模型转换过程中的 IR 导出成 float tflite 模型，可以先确认导出的 float tflite 结果与原始平台结果是否保持一致。

导出方式：

- 设置环境变量 export ADLA_EXPORT_MIDDLE_TO_TFLITE =True
- 模型转换时，量化类型选择 float32，例如：--quantize-dtype float32
- 此时输出目录下会保存出 float tflite 模型，可以跟原始模型进行结果比对
- 注意如果原始模型是 nchw 格式，注意在测试 float tflite 时，使用相同的前处理，然后再多做一次 transpose 即可

(2) 确认 float 结果和 quantized 的结果是否保持一致

将模型转换过程中的 IR 导出 float tflite 和 quantized tflite，确认 float 结果与 quantized 的结果是否保持一致。

Quantized 导出方式：

- a. 设置环境变量 `export ADLA_EXPORT_MIDDLE_TO_TFLITE =True`
- b. 模型正常进行转换即可

(3) 确认 quantized 的结果与板端是否保持一致

将 pc 侧的 quantized tflite 结果与板侧结果进行对比，注意保持输入一致性。板侧输出的数据格式跟 tflite 一样，是 NHWC 格式。

(4) 问题分析

- a. 如果第一步结果异常：确认输入一致性后，如果结果不一致，需将模型以及量化图片、量化参数信息提供给 Amlogic 进行 debug
- b. 如果第二步结果异常：第一步结果 ok，第二步结果异常，表明量化效果差。
 - 首先先 check 下量化参数 `channel-mean-value` 等是否正确，量化数据集 `dataset.txt` 中是否提供足够的图片，一般建议 200~500 张。
 - 如果当前使用的是 `int8 perlayer` 量化方式，可以尝试使用 `int8 perchannel`，如果相同的参数命令、相同的量化图片，`int8 perchannel` 结构正常，表明此模型有些 layer 使用 `perlayer` 量化不太友好，建议直接使用 `int8 perchannel` 量化方式
 - 如果当前使用的是 `int8 perchannel` 方式，可以尝试使用 `int16` 量化测试下
- c. 第三步结果异常：

底层 driver 或者 compiler 错误，此时需要将 quantized tflite 模型以及测试 quantized tflite 对应的输入输出数据提供给 Amlogic 进行底层问题的 debug。